

WEEK 3

Machine Learning Foundations

Practical ML • Model Evaluation • Business-Driven Model Selection

Core Goal

Build, evaluate and compare traditional ML models professionally.

Central lesson: The best model is not automatically the model with the highest accuracy. Model usefulness depends on the business problem, the cost of mistakes, the evaluation metric, and engineering trade-offs.

1. ML Problem Formulation

Question: Hum predict kya karna chahte hain?

```
Input features
↓
Model
↓
Target / Prediction
```

Features (X)

Wo input variables jin se model prediction learn karta hai.

```
X = df[["age", "salary", "job_level"]]
```

Label / Target (y)

Wo value jo model ko predict karni hai.

```
y = df["left_company"]
```

2. Classification vs Regression

Type	Output	Examples
Classification	Category / class	Spam/Not Spam, Churn/Stay, Fraud/Normal
Regression	Continuous number	House price, Salary, Temperature, Watch hours

Shortcut: Category → Classification Continuous number → Regression

3. Linear Regression

Regression problems ke liye use hota hai. Example: experience → salary.

```
Salary = weight × experience + bias
```

Goal: input aur continuous output ke darmiyan relationship learn karna.

4. Logistic Regression

Naam mein regression hai, lekin classification ke liye use hota hai. Model probability de sakta hai, for example 0.85 = estimated 85% probability of churn.

```
probability >= threshold
↓
churn
```

Common default threshold: 0.5. Threshold business requirement ke according tune kiya ja sakta hai.

5. Decision Trees

Decision Tree data ko questions/splits ke through divide karta hai.

```
Contract = Monthly?
|
Yes
↓
Support calls > 5?
|
Yes
↓
Churn
```

Advantages: easy to understand, rules ki form mein explainable, numerical aur categorical patterns handle kar sakta hai.

Problem: Decision Trees easily overfit kar sakte hain.

6. Random Forest

Random Forest multiple Decision Trees ka ensemble hai.

```
Tree 1
Tree 2
Tree 3
...
Tree N
↓
Combined prediction
```

Advantages: single tree se generally more stable, complex relationships learn kar sakta hai, generalization improve ho sakti hai.

Trade-offs: single tree se less interpretable, more computation aur memory.

7. Overfitting & Underfitting

Overfitting

Model training data ko bohat closely learn/remember karta hai aur unseen data par poor perform karta hai.

```
Training accuracy = 99%
Validation accuracy = 65%
```

Large train/validation gap overfitting ka strong signal hai.

Possible solutions: more data, regularization, irrelevant features remove karna, model complexity reduce karna, tree ke liye max_depth reduce karna, min_samples_split increase karna.

Underfitting

Model data ke patterns ko sufficiently learn nahi kar pa raha.

```
Training accuracy = 60%
Validation accuracy = 58%
```

Possible solutions: more expressive model, better features, appropriate complexity.

8. Train/Test Split

```
X_train, X_test, y_train, y_test = train_test_split(
    X,
    y,
    test_size=0.2,
    random_state=42,
    stratify=y
)
```

Training set: model learn karta hai. **Test set:** unseen data par final performance check hota hai.

Example: 10,000 examples → 8,000 training + 2,000 testing.

9. Stratified Split

Classification mein class imbalance ho sakta hai. **stratify=y** train/test splits mein class distribution ko approximately maintain karta hai.

```
Original: 90% Stay / 10% Churn
Train: 90% Stay / 10% Churn
Test: 90% Stay / 10% Churn
```

Important: Stratification data ko artificially 50/50 balance nahi karti; original ratio maintain karti hai.

10. Preprocessing

Real-world data mein missing values, numerical features aur categorical features hote hain.

Numerical Features

```
SimpleImputer(strategy="median")
StandardScaler()
```

Median extreme values ka effect mean ke comparison mein kam karta hai. Scaling numerical values ko comparable scale par laati hai.

Categorical Features

```
SimpleImputer(strategy="most_frequent")
OneHotEncoder()
```

Example: Monthly, Yearly, Two-year ko separate encoded columns mein convert kiya ja sakta hai.

11. Pipelines

```
Raw Data
↓
Imputer
↓
Scaler / Encoder
↓
Model
↓
Prediction
```

```
model_pipeline = Pipeline([
    ("preprocessor", preprocessor),
    ("model", LogisticRegression())
])
```

Benefits: consistent preprocessing, fewer manual mistakes, training aur inference mein same transformations, easier deployment.

12. ColumnTransformer

```
preprocessor = ColumnTransformer([
    ("numeric", numeric_pipeline, numeric_features),
    ("categorical", categorical_pipeline, categorical_features)
])
```

Different column groups par different preprocessing apply karne ke liye use hota hai.

13. Confusion Matrix

Predicted
0 1
Actual 0 TN FP
Actual 1 FN TP

Term	Meaning
TP	Actual positive + predicted positive
FP	Actual negative + predicted positive (false alarm)
FN	Actual positive + predicted negative (missed positive)
TN	Actual negative + predicted negative

14. Accuracy

Accuracy = correct predictions / total predictions

Formula: $(TP + TN) / (TP + TN + FP + FN)$

Warning: Imbalanced data mein accuracy misleading ho sakti hai. Example: 99% normal + 1% fraud; always predicting normal gives ~99% accuracy but detects no fraud.

15. Precision

Question: **Model ne jitne positive predict kiye, unmein se kitne actually positive thay?**

Formula: $TP / (TP + FP)$

High precision useful hoti hai jab false positives expensive hon.

16. Recall

Question: **Actual positive cases mein se kitne model ne identify kiye?**

Formula: $TP / (TP + FN)$

High recall useful hoti hai jab false negatives dangerous/expensive hon: fraud, medical screening, churn.

17. Precision vs Recall

Business concern	Metric
False Positive dangerous / expensive	Precision
False Negative dangerous / expensive	Recall

Churn example: agar business maximum churners identify karna chahta hai, recall priority ho sakti hai.

18. F1 Score

Precision aur Recall ka harmonic mean.

Formula: $2 \times (\text{Precision} \times \text{Recall}) / (\text{Precision} + \text{Recall})$

Useful jab precision aur recall dono important hon aur balanced view chahiye.

19. Class Imbalance

Example: 90% Stay, 10% Churn. Model majority class ki taraf biased ho sakta hai.

Stratification

```
stratify=y
```

Class Weight

```
class_weight="balanced"
```

Minority class ko greater importance mil sakti hai. Common trade-off: recall ↑, false positives ↑, precision ↓.

20. Threshold Tuning

Classification model probability produce kar sakta hai.

```
Customer A → 0.82
Customer B → 0.61
Customer C → 0.35
Customer D → 0.12
```

Default threshold commonly 0.5 hota hai. Threshold lower karne se usually recall increase aur precision decrease hoti hai; threshold increase karne se usually precision increase aur recall decrease hoti hai.

```
threshold = 0.3
y_pred = (y_prob >= threshold).astype(int)
```

Week 3 project mein Random Forest ko lower threshold ke saath test kiya gaya aur recall ~93% tak gaya, lekin precision ~37% rahi. Ye precision/recall trade-off ka practical example tha.

21. Model Comparison

Same dataset par Logistic Regression, Decision Tree aur Random Forest compare kiye gaye.

Model	Typical strength	Trade-off
Logistic Regression	Fast, explainable, strong baseline	Complex non-linear patterns miss kar sakta hai
Decision Tree	Highly interpretable rules	Overfitting risk
Random Forest	More stable ensemble, complex patterns	Less interpretable, more computation

Important: Highest accuracy automatically best model nahi hota.

22. Business-Based Model Selection

Model choose karte waqt metrics ke saath engineering factors bhi consider karo:

Dimension	Question
Accuracy	Overall correctness kitni hai?
Precision	Positive predictions kitni reliable hain?
Recall	Actual positives mein se kitne pakre?
F1	Precision aur recall ka balance kaisa hai?
Explainability	Business/team ko decision samjha sakte hain?

Dimension	Question
Latency	Prediction kitni fast chahiye?
Maintenance	System kitna complex hai?
Business cost	FP aur FN ki actual cost kya hai?

23. Week 3 Churn Project

Dataset features:

```
age
monthly_spend
support_calls
tenure_months
contract_type

Target:
churn
```

Project workflow:

```
Dataset
↓
X / y
↓
Train/Test Split
↓
Preprocessing
↓
Pipeline
↓
Multiple Models
↓
Evaluation
↓
Class Weight
↓
Threshold Tuning
↓
Business-Based Model Selection
```

24. Important Code Patterns

X and y

```
X = df[
[
"age",
"monthly_spend",
"support_calls",
"tenure_months",
"contract_type"
]
]

y = df["churn"]
```

Split

```
X_train, X_test, y_train, y_test = train_test_split(
X,
y,
test_size=0.2,
random_state=42,
stratify=y
)
```

Numeric Pipeline

```
numeric_pipeline = Pipeline([
    ("imputer", SimpleImputer(strategy="median")),
    ("scaler", StandardScaler())
])
```

Categorical Pipeline

```
categorical_pipeline = Pipeline([
    ("imputer", SimpleImputer(strategy="most_frequent")),
    ("onehot", OneHotEncoder(handle_unknown="ignore"))
])
```

Complete Pipeline

```
model_pipeline = Pipeline([
    ("preprocessor", preprocessor),
    ("model", LogisticRegression())
])
```

25. Core Mental Models

Concept	Remember
Classification	Category → Classification
Regression	Continuous number → Regression
Precision	Jo positive bola, usmein kitne sahi?
Recall	Jo actual positive thay, unmein se kitne pakre?
Overfitting	Training performance high, unseen-data performance much lower
Class imbalance	Majority class dominates
Class weight	Minority-class mistakes ko more importance
Threshold ↓	Recall usually ↑, Precision usually ↓
Pipeline	Preprocessing + Model = one reproducible flow

26. Week 3 Most Important Takeaways

1. ML problem ko pehle correctly formulate karo.
2. Features (X) aur target (y) clearly identify karo.
3. Classification aur regression ka difference output type se samjho.
4. Accuracy ko blindly optimize mat karo.
5. Business requirement ke according metric choose karo.
6. False Positive aur False Negative ka business cost samjho.
7. Churn jaisi problem mein missed churners ke liye recall important ho sakta hai.
8. Class imbalance ko identify karo.
9. stratify=y se class distribution maintain karo.
10. class_weight="balanced" minority class ko greater importance de sakta hai.
11. Threshold tuning se precision/recall trade-off control kar sakte ho.
12. Decision Trees interpretable hain but overfit kar sakte hain.

13. Random Forest multiple trees combine karta hai aur generally more stable hota hai.
14. Pipelines preprocessing aur model ko consistent rakhti hain.
15. Best model = highest accuracy zaroori nahi.
16. Production decision mein metrics + business requirements + explainability + latency + maintenance sab consider karo.

27. Week 3 Core Workflow

```
Problem Formulation
↓
Features / Target
↓
Classification / Regression
↓
Train/Test Split
↓
Preprocessing
↓
Pipeline
↓
Train Multiple Models
↓
Confusion Matrix
↓
Precision / Recall / F1
↓
Class Imbalance
↓
Class Weight
↓
Threshold Tuning
↓
Business-Based Model Selection
```

Revision checkpoint: Agar tum ye workflow explain kar sakte ho aur har step ke “why” ko justify kar sakte ho, to Week 3 ki conceptual foundation strong hai.